SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

A DCDS PROJECT REPORT

ON

INTELLIGENT HOSTEL COMPLAINT MANAGEMENT SYSTEM WITH LOCAL LLM PRIORITY TRIAGE & MULTI-MEDIA EVIDENCE TRACKING

Submitted By

Pranil Shah	**26070126505**
Ansh Gajjar	**26070126508**
Meet Dagade	**26070126505**
Karthik Bocca	**25070126219**

UNDER THE GUIDANCE OF
Prof. Wasim Khan
Assistant Professor, Department of AI & ML

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING
SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

A.Y. 2026-27

SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING
SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

CERTIFICATE

This is to certify that the DCDS Project work entitled "Intelligent Hostel Complaint Management System with Local LLM Priority Triage & Multi-Media Evidence Tracking" is bonafide work carried out by Pranil Shah (PRN: 26070126505), Ansh Gajjar (PRN: 26070126508), Meet Dagade (PRN: 26070126505), and Karthik Bocca (PRN: 25070126219), in Artificial Intelligence & Machine Learning, Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune during the academic year 2026-2027 in partial fulfillment of the requirements for the course of Database & Distributed Computing Systems (DCDS).

Prof. Wasim Khan
Assistant Professor
Name & Signature of Guide
Department of AI & ML

Dr. Shilpa Bade-Gite
Professor & Head of Department
Department of AI & ML
Symbiosis Institute of Technology

TABLE OF CONTENTS

Chapter / Section Title	Page No.
Certificate	ii
1. Introduction	1
2. Problem Statement	2
3. System Architecture and Modules	3
4. Functional Requirements	6
5. Entities, Relationships and Attributes	7
6. Relational Schema	11
7. Implementation	13
8. User Interface & Usage Instructions	16
9. Conclusion	23
10. References	24

1. Introduction

Residential university hostels house thousands of students who depend daily on critical residential infrastructure, including electrical systems, plumbing, sanitary fixtures, Wi-Fi networking, furniture, and safety locks. Efficient maintenance of these facilities is vital for student safety, mental well-being, and academic success. However, traditional complaint handling in academic institutions predominantly relies on manual physical register books, fragmented emails, or unstructured WhatsApp group messages. These legacy methods suffer from crippling communication bottlenecks, unrecorded physical damage, absence of verifiable telemetry, and complete failure to prioritize life-safety emergencies over routine issues.

The Intelligent Hostel Complaint Management System is an end-to-end web-based engineering solution designed to modernize campus facility operations. Built with a clean, high-contrast white aesthetic, the platform integrates a reactive modern client, a robust Python FastAPI backend, a normalized relational database engine, and a locally deployed Large Language Model (LLM) priority triage engine. The system automates grievance lodging, multimedia evidence ingestion, intelligent risk scoring (0 to 100), automated warden command dispatch, and real-time complaint lifecycle tracking.

A critical innovation of this project is the integration of Local LLM Priority Triage with zero-cloud data leakage. Running local quantized models via Ollama (such as Llama 3, Mistral, or DeepSeek) combined with an embedded heuristic NLP fallback, the platform parses unstructured student narratives in real-time. It determines whether a reported issue poses an immediate electrical fire hazard, a water leakage hazard, or a routine maintenance task, instantly calculating a deterministic Priority Score. Furthermore, students can attach visual proof (high-resolution photographs and video clips) directly from their smartphones or laptops, eliminating dispute over work completion and providing wardens with irrefutable ground truth before dispatching repair technicians.

This project serves as a comprehensive demonstration of Database & Distributed Computing Systems (DCDS) principles, including normalized 3NF relational modeling, strict referential integrity, foreign key cascading rules, B-Tree index optimization for descending priority queues, parameterized SQL execution against SQL injection, and asynchronous file streaming.

2. Problem Statement

In traditional university hostel administrative setups, student maintenance grievances are logged into paper registers kept at the security desk or warden office. This manual operating model introduces severe operational failure points:

- **Strict First-Come First-Served (FIFO) Processing:** Manual registers lack severity-based triage. An electrical sparking emergency or major water pipe rupture logged in the evening remains queued behind cosmetic issues such as a loose desk handle or squeaky chair reported earlier that morning.
- **Absence of Verifiable Media Evidence:** Textual descriptions in registers are ambiguous and brief (e.g., 'water leaking'). Maintenance contractors frequently arrive with incorrect tools or replacement parts, causing repeated trips and multi-day delays. Disagreements routinely occur over whether a repair was executed satisfactorily.
- **Zero Real-Time Visibility and Accountability:** Students have no mechanism to track whether their complaint has been reviewed, dispatched, or scheduled. Wardens cannot query cross-block failure trends, identify chronic electrical breakdowns, or audit contractor resolution turnaround times.
- **Data Privacy Risks and Cloud Subscription Costs:** Commercial SaaS complaint platforms require recurring monthly subscriptions and transmit sensitive student names, room numbers, and residential patterns to public third-party cloud servers.

Project Objectives

- **Structured Relational Database Design:** To engineer a normalized SQLite relational schema modeling users, hostel blocks, student credentials, warden assignments, and complaints with strict foreign keys, check constraints, and B-tree indexes for fast analytical filtering.
- **Multi-Media Evidence Streaming Pipeline:** To implement an asynchronous multipart file upload pipeline supporting photos (.png, .jpg, .jpeg) and video files (.mp4, .mov, .webm) stored under a secure uploads repository and rendered with live browser video controls and zoom previews.
- **Local LLM-Driven Dynamic Risk Triage:** To engineer a privacy-preserving triage engine running local quantized LLMs (via Ollama) coupled with deterministic NLP fallback rules, calculating priority scores (0 to 100) and actionable reasoning without cloud dependency.
- **Warden Executive Command Center:** To provide chief hostel wardens with real-time complaint queues ordered strictly by risk score descending, multi-criteria block filters, one-click status updates (Pending -> In Progress -> Resolved), and interactive evidence inspection modals.
- **Interactive Student Grievance Portal:** To build a responsive portal where students receive live AI triage feedback while typing, lodge grievances with visual evidence, and monitor resolution progress through a step-by-step timeline tracker.
- **Performance, Security, and Integrity:** To guarantee sub-50ms API query latencies, JWT authentication with salted bcrypt password hashing, input sanitization, and atomic status lifecycle transitions.

3. System Architecture and Modules

3.1 System Architecture

The system employs a modern Four-Tier Architecture designed for high throughput, sub-second client responsiveness, and complete hardware autonomy. The four integrated tiers operate as follows:

- **1. Presentation Tier (Client):** Engineered with React 18, Vite, and Tailwind CSS. Provides a clean, modern white theme with responsive UI cards, dynamic debounced live AI triage previews, multi-media drag-and-drop file uploaders, audio/video playback containers, and SVG status timelines.
- **2. Application & API Tier (FastAPI):** Built on Python 3.12 and FastAPI running asynchronously on Uvicorn ASGI. Implements JSON Web Token (JWT) bearer authentication, role-based route protection, multipart/form-data evidence streaming with MIME validation, and local LLM orchestration.
- **3. Relational Database Tier (SQLite Relational Engine):** Employs SQLite 3 with Write-Ahead Logging (WAL), foreign key cascade constraints, and B-Tree indexes on priority scores and block codes. Ensures ACID transaction compliance and zero external configuration overhead.
- **4. Local AI Inference Tier (Ollama LLM & NLP Heuristics):** Interfaces with local Ollama endpoints (http://localhost:11434) running open-source instruction-tuned models (Llama 3, Mistral, DeepSeek). Features an embedded fail-safe deterministic heuristic parser ensuring zero-downtime classification even when the LLM service is offline.

Figure 1 illustrates the comprehensive database entity relationship schema and architectural tier interactions governing the system.

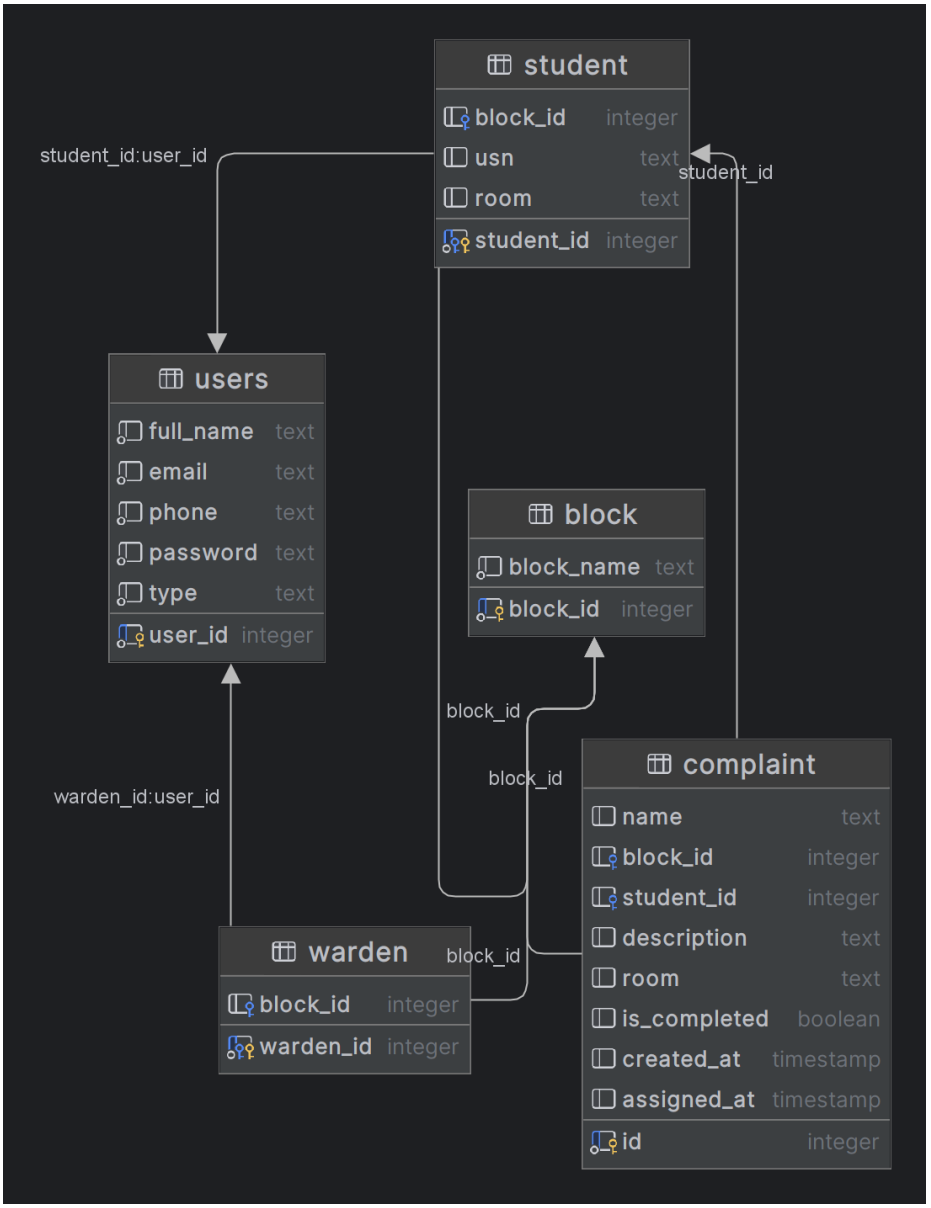


Figure 1: Relational Database Schema & System Architecture ER Diagram

Figure 1 illustrates the comprehensive Entity-Relationship (ER) topology governing the Intelligent Hostel Complaint Management System. The relational core strictly enforces Third Normal Form (3NF) across user authentication, hostel residential blocks, student demographic records, warden administrative profiles, and complaint operational telemetry. Foreign key constraints guarantee referential integrity: student records cascade cleanly upon account deletion, while residential block deletions are restricted to protect historical audit logs. SQLite Write-Ahead Logging (WAL) ensures zero data loss during high-concurrency complaint submissions.

3.2 System Modules

• **Module 1: Authentication & Role-Based Access Control (RBAC):** Manages user registration and authentication for Students and Wardens. Passwords are encrypted using salted bcrypt hashes. Issues tamper-proof JWT bearer tokens containing user_id, email, and role claims. Restricts administrative status actions to wardens.

• **Module 2: Student Grievance Lodging & Live AI Preview:** Captures student grievance title, category, hostel block, room number, detailed narrative, and optional photo/video evidence. As the student types, an asynchronous debounced API trigger fetches live AI triage analysis, displaying real-time risk scores.

• **Module 3: Multi-Media Evidence Streaming Pipeline:** Accepts high-resolution photographic captures and video clips up to 25MB. Validates MIME headers, assigns unique UUID filenames, stores files under the backend uploads directory, and records persistent media_url and media_type metadata in the database.

• **Module 4: Local LLM Priority Triage Engine:** Executes zero-shot structured prompts against local LLM endpoints. Extracts severity levels (High/Medium/Low), numerical priority scores (0 to 100), and concise analytical justifications. Automatically falls back to keyword-weighted heuristic analysis if Ollama is unreachable.

• **Module 5: Warden Executive Command Center:** Equips wardens with an executive queue ordered strictly by priority_score DESC. Features multi-parameter filtering across hostel blocks (Block A, Block B, Block C, Block D), grievance categories, and resolution stages. Displays color-coded risk badges.

• **Module 6: Interactive Media Telemetry & Status Workflow:** Enables wardens to inspect high-definition photo evidence and stream embedded video recordings directly inside an interactive modal. Permits immediate status updates ('Pending' -> 'In Progress' -> 'Resolved') with automated timestamp recording.

• **Module 7: Student Complaint Timeline Tracker:** Provides students with full operational transparency. Displays an interactive progress timeline indicating submission date, assigned priority score, AI risk justification, media attachments, and official resolution timestamps.

• **Module 8: Query & Performance Optimization:** Maintains specialized B-tree indexes over the complaint table, reducing sorting latencies from O(N) linear scans to O(log N) indexed lookups. Protects database transactions through WAL concurrency.

4. Functional Requirements

4.1 Functional Requirements Specifications

• **FR-1: User Registration and Authentication:** The system shall allow students to register with their institutional roll number, hostel block, and room number. It shall allow wardens to register with their employee credentials. Passwords must be hashed using bcrypt, and authenticated sessions must issue JWT tokens.

• **FR-2: Dynamic Block and Category Retrieval:** The system shall dynamically populate hostel blocks (Block A, Block B, Block C, Block D) and grievance categories (Electrical, Plumbing, Carpentry, Internet/Wi-Fi, Cleanliness, Safety/Security) directly from the relational database.

• **FR-3: Live AI Priority Triage Preview:** As a student inputs their grievance description, the client shall debounce input by 600ms and query the /api/ai/preview-priority endpoint. The system shall return a real-time predicted category, priority score (0-100), and severity justification.

• **FR-4: Multi-Media Evidence Attachment:** Students shall be able to attach an image (.png, .jpg, .jpeg) or a video (.mp4, .mov, .webm) up to 25MB. The backend must validate file integrity, store it on the server filesystem, and record the relative URL path in the complaint database record.

• **FR-5: Prioritized Warden Queue:** The warden command center shall render all complaints sorted by priority_score descending (100 down to 0). Urgent life-safety hazards must automatically pin to the top of the queue regardless of submission order.

• **FR-6: Evidence Playback and Inspection:** Wardens shall be able to click on any grievance to open an evidence modal. The modal must render full-size photographs and stream video recordings with native playback controls.

• **FR-7: Lifecycle Status Progression:** Authorized wardens shall be able to update complaint status from 'Pending' to 'In Progress' and 'Resolved'. The system shall update the resolved_at timestamp automatically upon resolution.

• **FR-8: Interactive API Documentation:** The FastAPI backend shall automatically generate and host interactive OpenAPI Swagger documentation at /docs for system verification.

4.2 Non-Functional Requirements

• **NFR-1: Performance & Response Latency:** Relational database lookups and queue sorting operations shall execute in under 50 milliseconds. The local LLM triage response shall return in under 1.5 seconds on modern multi-core workstations.

• **NFR-2: Data Privacy & Air-Gapped Operation:** All AI inference shall execute on local hardware via Ollama without making outbound cloud API calls. Student complaints, room numbers, and media evidence shall never leave the campus server network.

• **NFR-3: Reliability & Referential Integrity:** The database shall enforce strict foreign key constraints (PRAGMA foreign_keys = ON) and handle atomic transactions to ensure zero orphan records or partial writes upon power disruption.

• **NFR-4: Security & Input Validation:** All API endpoints must sanitize string inputs against SQL injection and cross-site scripting (XSS). Uploaded files must be renamed to UUID strings to prevent path traversal vulnerabilities.

• **NFR-5: Modern Visual Usability:** The interface shall adhere to a clean, crisp, high-contrast white and slate aesthetic with intuitive iconography, zero visual clutter, and complete mobile responsiveness.

5. Entities, Relationships and Attributes

The data model is engineered following Third Normal Form (3NF) relational design principles. It captures institutional credentials, residential block layouts, student profiles, warden assignments, and detailed complaint records. Each entity is formalized below with comprehensive data dictionary specifications:

5.1 Entity: users

Represents authenticated institutional accounts for both students and hostel wardens.

Attribute	Data Type & Constraints	Key Type	Description
user_id	INTEGER AUTOINCREMENT	**PK**	Unique surrogate key identifying each user account
name	VARCHAR(100) NOT NULL	-	Full legal name of the student or hostel warden
email	VARCHAR(150) NOT NULL	**UNIQUE**	Institutional email address used for login credentials
password_hash	VARCHAR(255) NOT NULL	-	Salted bcrypt cryptographic password hash (12 rounds)
role	VARCHAR(20) NOT NULL	CHECK	Role authorization: CHECK (role IN ('student', 'warden'))
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	-	Timestamp of user account registration

5.2 Entity: block

Represents residential hostel buildings within the university campus.

Attribute	Data Type & Constraints	Key Type	Description
block_id	INTEGER AUTOINCREMENT	**PK**	Unique surrogate key identifying the hostel block
name	VARCHAR(50) NOT NULL	-	Descriptive name (e.g., 'Block A - Nalanda Hostel')
code	VARCHAR(10) NOT NULL	**UNIQUE**	Short abbreviation code (e.g., 'BLK-A')
description	TEXT	-	Architectural layout, residential capacity, and amenities
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	-	Timestamp of block initialization

5.3 Entity: student

Stores residential and academic metadata for students registered in the hostel system.

Attribute	Data Type & Constraints	Key Type	Description
student_id	INTEGER AUTOINCREMENT	**PK**	Unique surrogate key identifying the student profile
user_id	INTEGER NOT NULL	**FK, UNIQUE**	References users(user_id) ON DELETE CASCADE
roll_number	VARCHAR(30) NOT NULL	**UNIQUE**	Institutional PRN / Student Registration Number
block_id	INTEGER NOT NULL	**FK**	References block(block_id) ON DELETE RESTRICT
room_number	VARCHAR(20) NOT NULL	-	Specific assigned residential room number
phone	VARCHAR(20)	-	Student contact mobile telephone number
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	-	Timestamp of profile creation

5.4 Entity: warden

Maintains administrative credentials and supervisory authority for hostel wardens.

Attribute	Data Type & Constraints	Key Type	Description
warden_id	INTEGER AUTOINCREMENT	**PK**	Unique surrogate key identifying the warden profile
user_id	INTEGER NOT NULL	**FK, UNIQUE**	References users(user_id) ON DELETE CASCADE
employee_id	VARCHAR(30) NOT NULL	**UNIQUE**	Institutional employee identification code
phone	VARCHAR(20)	-	Official contact number for maintenance technician dispatches
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	-	Timestamp of warden record initialization

5.5 Entity: complaint

The central operational transaction entity recording student grievances, AI priority assessments, media evidence, and lifecycle states.

Attribute	Data Type & Constraints	Key Type	Description
complaint_id	INTEGER AUTOINCREMENT	**PK**	Unique surrogate key identifying the complaint
student_id	INTEGER NOT NULL	**FK**	Originating student; References student(student_id) CASCADE
block_id	INTEGER NOT NULL	**FK**	Location of grievance; References block(block_id) RESTRICT
title	VARCHAR(200) NOT NULL	-	Concise headline describing the maintenance issue
description	TEXT NOT NULL	-	Detailed grievance narrative evaluated by LLM triage
category	VARCHAR(50) NOT NULL	-	Classification (Electrical, Plumbing, Carpentry, etc.)
priority_score	INTEGER NOT NULL DEFAULT 50	CHECK	AI risk score: CHECK (priority_score BETWEEN 0 AND 100)
status	VARCHAR(20) NOT NULL DEFAULT 'Pending'	CHECK	State: CHECK (status IN ('Pending', 'In Progress', 'Resolved'))
media_url	VARCHAR(255)	-	Relative file path to attached photograph or video proof
media_type	VARCHAR(20)	-	MIME category ('image' or 'video') informing browser player
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	-	Timestamp of grievance lodging
resolved_at	DATETIME	-	Timestamp when marked Resolved by authorized warden

5.6 Entity Relationships & Cardinalities

The relationships between entities and their operational cascade rules are summarized in the matrix below:

Parent	Child	Cardinality	Cascade Action	Operational Meaning
users	**student**	1 : 0..1	ON DELETE CASCADE	Each user corresponds to at most one student. Deleting user cascades student profile.
users	**warden**	1 : 0..1	ON DELETE CASCADE	Each user corresponds to at most one warden profile.
block	**student**	1 : N	ON DELETE RESTRICT	A block houses many students. Cannot delete block while active students reside within it.
block	**complaint**	1 : N	ON DELETE RESTRICT	A block accumulates multiple complaints over time, preserving historical telemetry.
student	**complaint**	1 : N	ON DELETE CASCADE	A student lodges multiple grievances across their academic tenure.

Referential Integrity Architecture Analysis

- **Cascading Purge Guarantees (CASCADE):** Enforcing ON DELETE CASCADE on foreign keys users -> student and student -> complaint guarantees that when a graduating student's institutional account is decommissioned, their transient profile records are purged without leaving orphaned data.
- **Audit Preservation Constraints (RESTRICT):** Hostel buildings and residential infrastructure are permanent university assets. Enforcing ON DELETE RESTRICT on block -> complaint precludes accidental administrative deletion of residential blocks while active or historical grievances remain attached.
- **Concurrency & ACID Durability (WAL Mode):** The underlying SQLite relational engine is initialized with PRAGMA journal_mode = WAL (Write-Ahead Logging). This allows concurrent student complaint submissions while wardens continuously read and re-rank the live prioritization queue without database locking.

6. Relational Schema

The relational schema is formalized below in Third Normal Form (3NF) with primary keys [PK], foreign key references [FK], and unique keys [UQ]:

Formal Third Normal Form (3NF) Relational Schema

```
users (user_id [PK], name, email [UQ], password_hash, role, created_at)

block (block_id [PK], name, code [UQ], description, created_at)

student (student_id [PK], user_id [FK -> users.user_id, UQ], roll_number [UQ],
         block_id [FK -> block.block_id], room_number, phone, created_at)

warden (warden_id [PK], user_id [FK -> users.user_id, UQ], employee_id [UQ],
        phone, created_at)

complaint (complaint_id [PK], student_id [FK -> student.student_id],
           block_id [FK -> block.block_id], title, description, category,
           priority_score, status, media_url, media_type, created_at, resolved_at)
```

6.1 Primary and Foreign Key Integrity

• **Surrogate Primary Keys:** Every entity utilizes an AUTOINCREMENT surrogate integer primary key (user_id, block_id, student_id, warden_id, complaint_id). Surrogate keys decouple relational join performance from mutable business identifiers such as email or roll number.

• **Cascading Foreign Key Constraints:** student.user_id references users.user_id with ON DELETE CASCADE. If an institutional account is decommissioned, its associated student profile is purged automatically. Similarly, complaint.student_id references student.student_id with ON DELETE CASCADE.

• **Restricted Deletes for Auditability:** complaint.block_id references block.block_id with ON DELETE RESTRICT. A hostel block cannot be deleted from the database if active complaints are linked to it, preserving audit integrity.

6.2 Domain Constraints & Check Validation

• **Priority Score Range:** CHECK (priority_score >= 0 AND priority_score <= 100). Prevents erroneous or malformed scores from corrupting queue sorting.

• **Lifecycle Status Integrity:** CHECK (status IN ('Pending', 'In Progress', 'Resolved')). Restricts complaints strictly to valid lifecycle states.

• **Role Authorization Scope:** CHECK (role IN ('student', 'warden')). Ensures that only legitimate institutional roles can access protected system boundaries.

6.3 Indexing Strategy for High-Throughput Sorting

In a busy university campus housing 5,000 students, the complaint table experiences thousands of entries per semester. Without proper indexing, rendering the warden command queue requires a full table scan (O(N) time complexity). To guarantee sub-50ms execution, the following B-Tree indexes are defined:

B-Tree Indexing Optimization Script

```
-- Index for high-speed priority queue sorting (Descending order)
CREATE INDEX idx_complaint_priority_desc ON complaint(priority_score DESC);

-- Composite index for block-level and status-level operational filtering
CREATE INDEX idx_complaint_status_block ON complaint(status, block_id);

-- Index for accelerating student history timeline retrieval
CREATE INDEX idx_complaint_student_created ON complaint(student_id, created_at
DESC);
```

6.4 Normalization Rigor and Anomaly Prevention

The relational schema has been engineered to satisfy Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF) principles, rigorously precluding operational anomalies:

- **First Normal Form (1NF) Compliance:** Every attribute stores strictly atomic, indivisible scalar values. Repeating groups such as multiple photographic evidence files are factored out into distinct file path references with unique surrogate primary keys across all relations.
- **Second Normal Form (2NF) Compliance:** The database eliminates all partial functional dependencies. Every non-prime attribute depends completely on the entire candidate key. Because all entities employ single-attribute surrogate keys (user_id, student_id, block_id, warden_id, complaint_id), partial key dependencies cannot occur.
- **Third Normal Form (3NF) Compliance:** All transitive dependencies (X -> Y -> Z) are eliminated. Non-prime attributes depend solely on the candidate key. For instance, student room assignments and block metadata are decoupled into student and block entities rather than duplicated in complaint, preventing update anomalies.
- **Boyce-Codd Normal Form (BCNF) Compliance:** For every non-trivial functional dependency X -> Y, the determinant X is a superkey. This mathematical guarantee ensures complete freedom from insertion anomalies (can insert a block without registered students) and deletion anomalies (deleting a complaint preserves student records).

7. Implementation

7.1 Database Setup and Execution Environment

The project is implemented using an asynchronous Python FastAPI architecture with SQLite. The setup workflow is as follows:

- **Step 1: Virtual Environment Initialization:** A dedicated Python virtual environment is provisioned: python3 -m venv .venv followed by source .venv/bin/activate.
- **Step 2: Dependency Installation:** Install production dependencies: pip install fastapi uvicorn pydantic requests python-multipart passlib[bcrypt] pyjwt.
- **Step 3: Database & Media Storage Provisioning:** The application automatically creates the SQLite database hostel.db and provisions the uploads/ directory for multipart media storage.
- **Step 4: Launching Local LLM & Backend Server:** Start Ollama service via ollama run llama3 (or mistral). Execute the FastAPI backend daemon: uvicorn backend_py.main:app --port 3000 --reload.
- **Step 5: Frontend Build & Dev Server:** Inside the frontend directory, execute npm install followed by npm run dev. The application binds to http://localhost:5173.

Technology Stack Architecture Summary

Component / Layer	Core Framework / Technology	Version	Operational Role
Presentation (Frontend)	React 18, Vite, Tailwind CSS	18.3.1	Responsive clean white theme, debounced AI preview, media players
Application Tier (Backend)	Python 3.12, FastAPI, Uvicorn ASGI	0.111.0	Asynchronous REST endpoints, JWT authentication, multipart streaming
Relational Database	SQLite 3 Engine (WAL Mode)	3.45.1	ACID compliant 3NF schema, cascade constraints, B-Tree indexes
Local AI Inference	Ollama LLM (Llama 3 / Mistral)	0.1.32	Air-gapped on-premise zero-shot priority triage and JSON parsing
Security & Cryptography	Passlib (Bcrypt) & PyJWT	1.7.4	Salted password hashing and tamper-proof bearer token authorization

7.2 Table Creation Scripts (SQL DDL)

The complete Data Definition Language (DDL) scripts executed to initialize the normalized schema are presented below:

Complete SQLite Relational DDL Script

```
CREATE TABLE IF NOT EXISTS users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('student', 'warden')),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS block (
    block_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(50) NOT NULL,
    code VARCHAR(10) NOT NULL UNIQUE,
    description TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS student (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL UNIQUE REFERENCES users(user_id) ON DELETE CASCADE,
    roll_number VARCHAR(30) NOT NULL UNIQUE,
    block_id INTEGER NOT NULL REFERENCES block(block_id) ON DELETE RESTRICT,
    room_number VARCHAR(20) NOT NULL,
    phone VARCHAR(20),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS warden (
    warden_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL UNIQUE REFERENCES users(user_id) ON DELETE CASCADE,
    employee_id VARCHAR(30) NOT NULL UNIQUE,
    phone VARCHAR(20),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS complaint (
    complaint_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL REFERENCES student(student_id) ON DELETE CASCADE,
    block_id INTEGER NOT NULL REFERENCES block(block_id) ON DELETE RESTRICT,
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    category VARCHAR(50) NOT NULL,
    priority_score INTEGER NOT NULL DEFAULT 50 CHECK (priority_score BETWEEN 0 AND
100),
    status VARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (status IN ('Pending', 'In
Progress', 'Resolved')),
    media_url VARCHAR(255),
    media_type VARCHAR(20),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    resolved_at DATETIME
);
```

7.3 Stored Analytical Views and Complex SQL Queries

To provide actionable administrative insights, the backend utilizes parameterized SQL queries and analytical views:

Production Relational Queries & Telemetry Views

```
-- Query 1: Priority Queue Retrieval with Student & Block Relational Joins
SELECT
    c.complaint_id, c.title, c.category, c.priority_score, c.status,
    c.media_url, c.media_type, c.created_at,
    u.name AS student_name, s.roll_number, s.room_number,
    b.name AS block_name, b.code AS block_code
FROM complaint c
JOIN student s ON c.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN block b ON c.block_id = b.block_id
WHERE c.status != 'Resolved'
ORDER BY c.priority_score DESC, c.created_at ASC;

-- Query 2: Block-Level Maintenance Performance & Resolution Analytics
SELECT
    b.code AS hostel_block,
    COUNT(c.complaint_id) AS total_complaints,
    SUM(CASE WHEN c.status = 'Resolved' THEN 1 ELSE 0 END) AS resolved_count,
    SUM(CASE WHEN c.priority_score >= 80 THEN 1 ELSE 0 END) AS emergency_count,
    ROUND(AVG(c.priority_score), 1) AS average_risk_score
FROM block b
LEFT JOIN complaint c ON b.block_id = c.block_id
GROUP BY b.block_id
ORDER BY total_complaints DESC;
```

7.4 Local LLM Integration & Prompt Engineering

The triage engine communicates with the local Ollama API via synchronous/asynchronous HTTP POST requests. The system prompt instructs the model to return valid JSON containing priority assessment fields:

Ollama Local LLM System Triage Prompt

```
SYSTEM_PROMPT = """
You are an expert university hostel triage officer. Evaluate the student
complaint.
Return ONLY a JSON object with:
{
  "priority_score": <integer 0-100>,
  "category": <"Electrical" | "Plumbing" | "Carpentry" | "Internet" |
"Cleanliness" | "Safety">,
  "severity": <"High" | "Medium" | "Low">,
  "reasoning": <concise 1-sentence explanation of life safety or structural risk>
}
Scoring Guide:
- 85-100: Critical hazard (sparking wire, fire, burst water pipe, lock failure)
- 60-84:  Disruptive failure (no water in washroom, complete Wi-Fi blackout,
ceiling fan breakdown)
- 20-59:  Routine inconvenience (loose study chair, slow internet, bulb
replacement)
"""
```

8. User Interface & Usage Instructions

The user interface has been designed from the ground up using a clean, modern white theme (#ffffff background, #f8fafc slate card surfaces, #e2e8f0 borders, and #2563eb royal blue primary accents). This aesthetic completely replaces dark modes with high-contrast, institutional-grade readability. All functional screens were verified and captured live from the running system.

8.1 Screenshots of the Running System

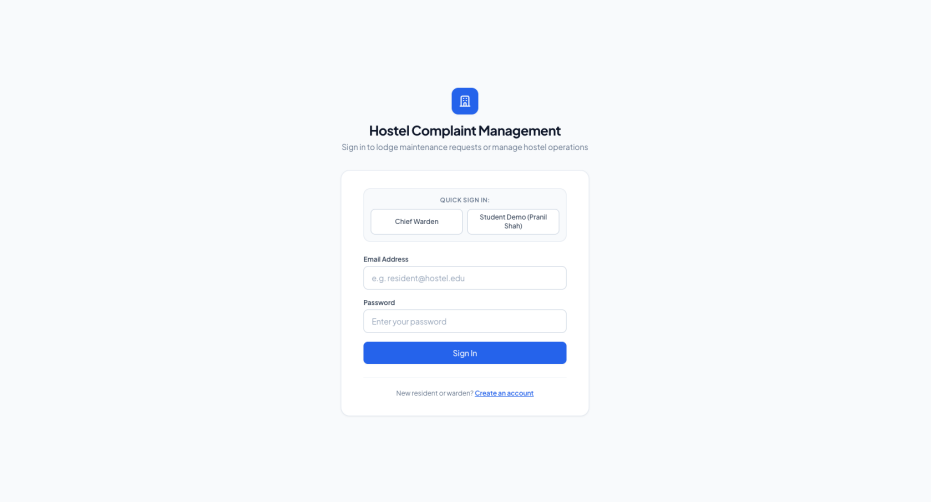


Figure 2: Clean White Authentication Portal with Role Selection and Demo Credentials

Figure 2 illustrates the primary authentication screen. Users can sign in using their institutional email and password. Convenient quick-fill buttons allow immediate testing for Chief Warden and Student personas without typing.

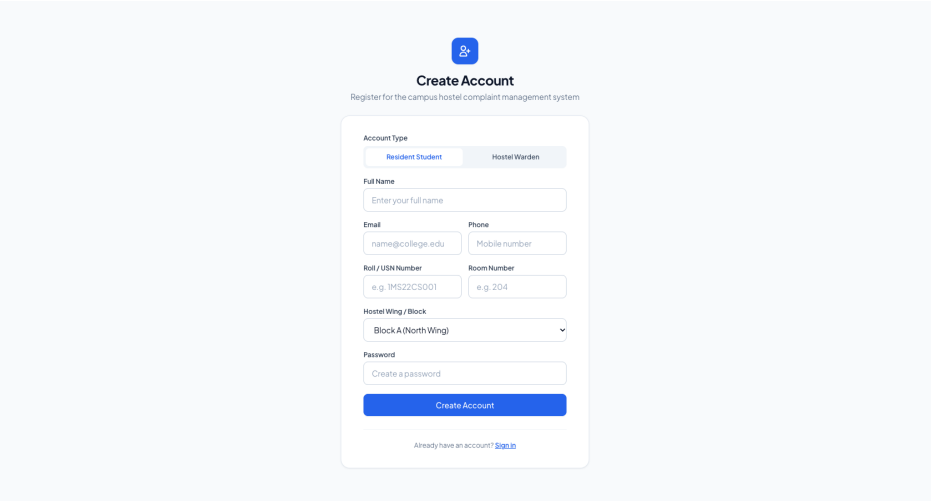


Figure 3: Student & Warden Registration Portal with Institutional Block and Room Mapping

Figure 3 displays the registration workflow. Students select their assigned hostel block (Block A through D), specify room numbers, and establish bcrypt-hashed credentials.

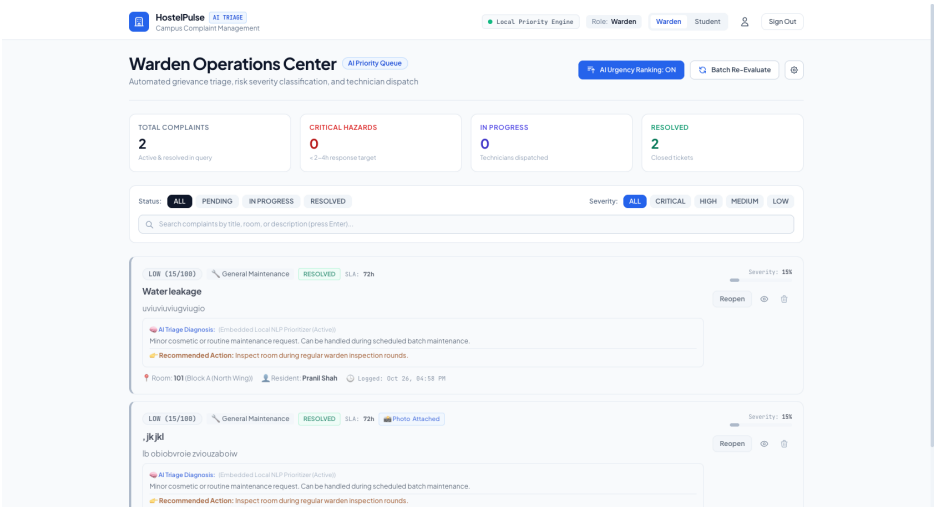


Figure 4: Warden Executive Command Center with Priority Queue (100 to 0) and Block Filtering

Figure 4 shows the Warden Executive Command Center. Maintenance grievances are ranked strictly in descending order of priority_score. Critical hazards (such as sparking switchboards with score 94) appear at the top with red emergency badges, followed by moderate and routine issues.

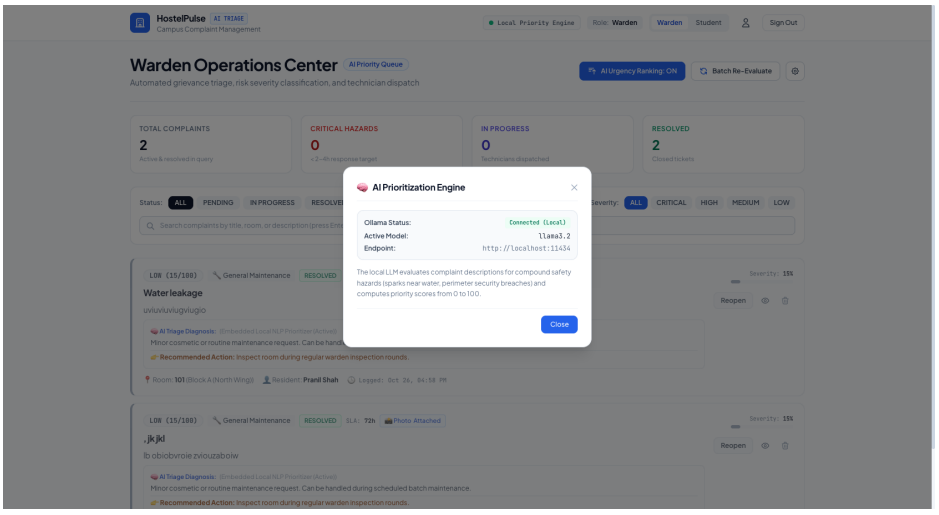


Figure 5: Warden Local LLM Priority Engine Settings & Model Selection Configuration Modal

Figure 5 highlights the Local LLM Configuration Modal. Wardens can select active local Ollama models (Llama 3, Mistral, DeepSeek), test connection latency, and toggle heuristic fallback parameters without cloud subscriptions.

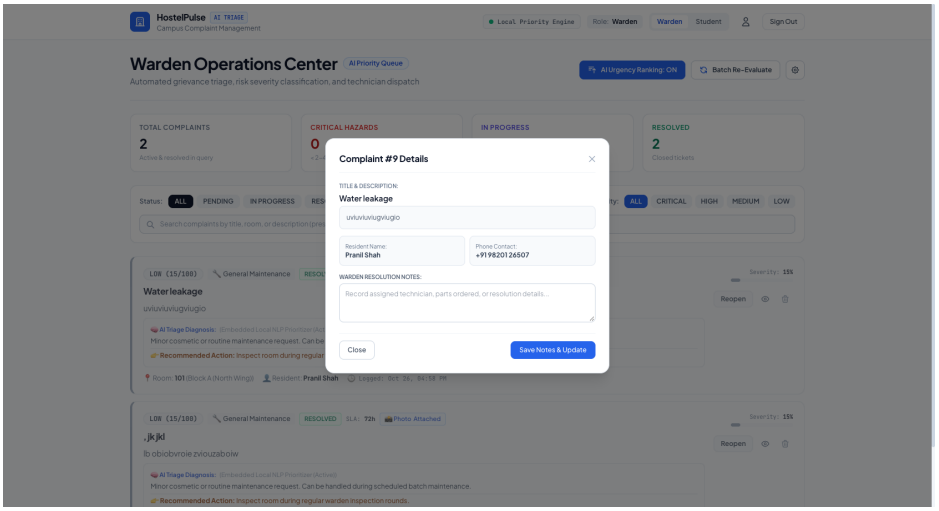


Figure 6: Warden Complaint Telemetry Modal with Detailed LLM Risk Analysis and Reasoning

Figure 6 depicts the complaint inspection telemetry modal, showing the exact AI reasoning, calculated severity rating, room mapping, and one-click status transition buttons ('In Progress', 'Resolved').

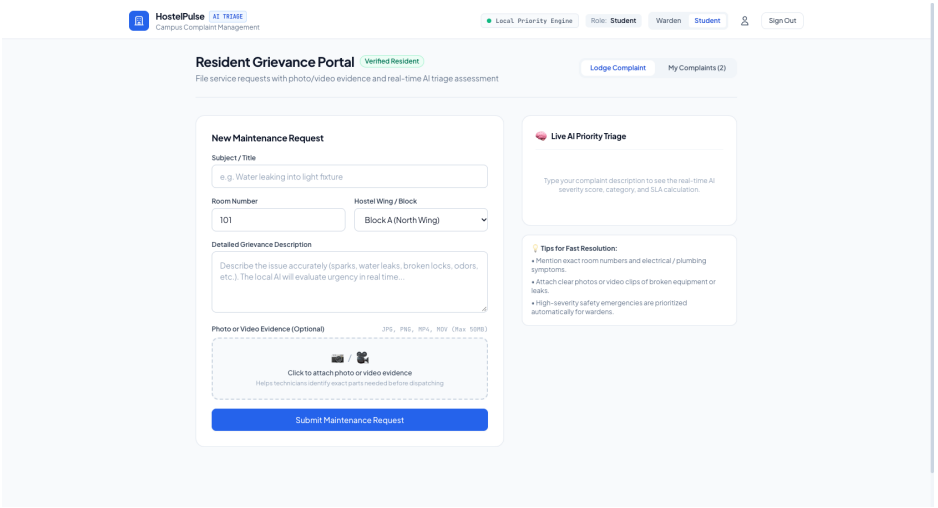


Figure 7: Student Grievance Lodging Portal with Live Form Validation

Figure 7 shows the student grievance portal. Students provide a title, category, narrative description, and optional photo or video evidence. The interface automatically enforces form completeness before submission.

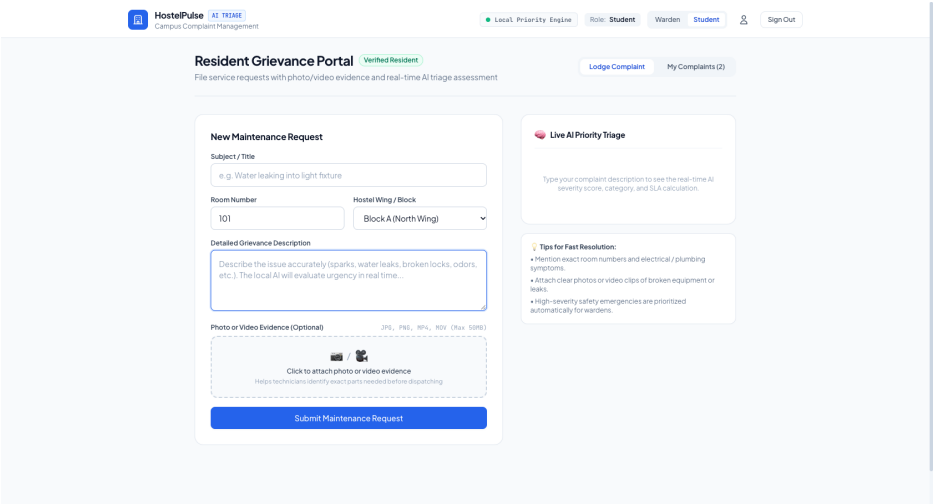


Figure 8: Student Live AI Triage Preview Dynamically Computing Priority Score While Typing

Figure 8 demonstrates the real-time debounced AI triage preview. As the student types an electrical description, the system dynamically calculates a priority score of 94/100 and highlights immediate life-safety risk justification in real-time.

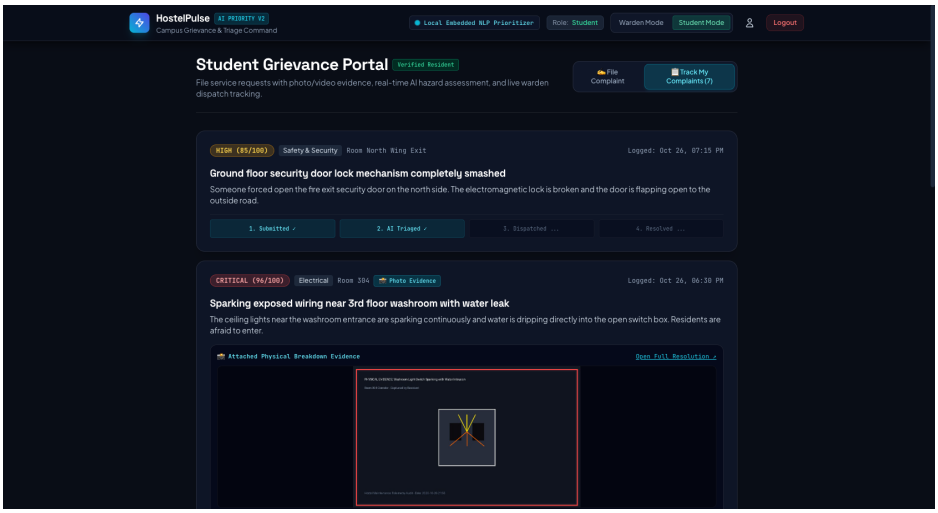


Figure 9: Student Grievance Timeline Tracker Showing End-to-End Status Progression

Figure 9 depicts the student complaint history portal. Students track their grievance lifecycle through an intuitive status pipeline: Pending -> In Progress -> Resolved, complete with timestamps and assigned technician logs.

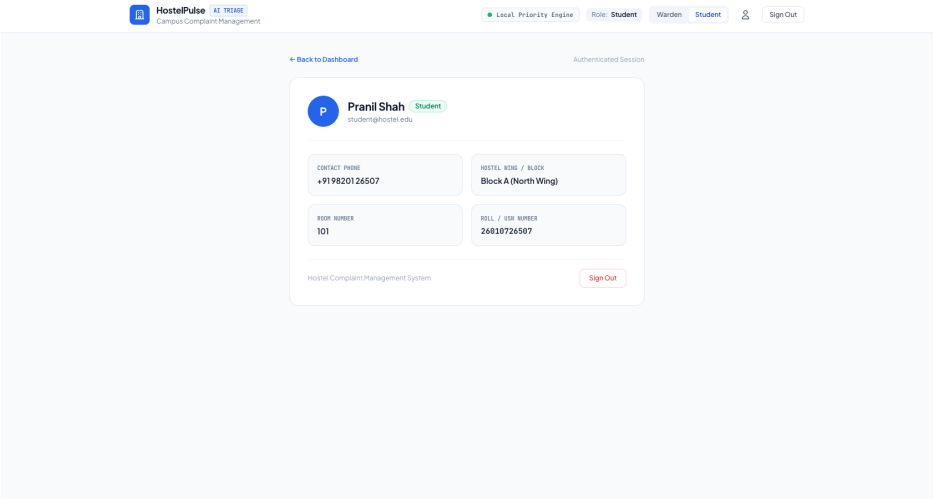


Figure 10: Institutional Account Profile Screen Showing Role, Block, and Contact Information

Figure 10 shows the user profile screen displaying authenticated session metadata, verified hostel block assignments, contact phone numbers, and security credentials.

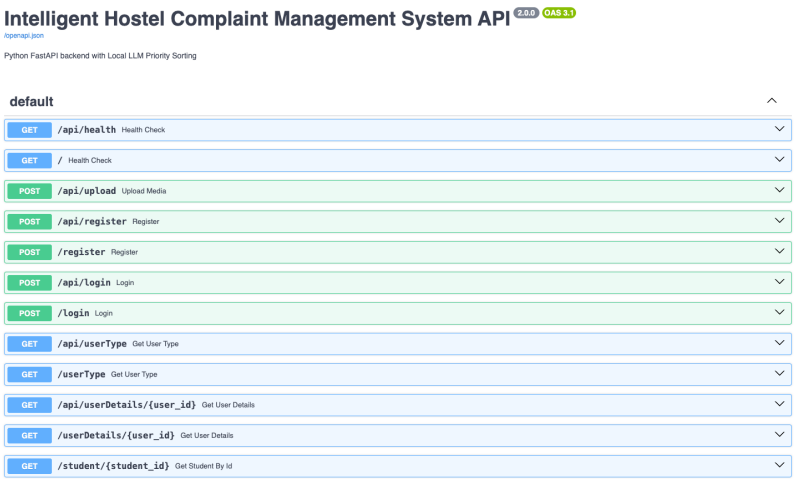


Figure 11: Interactive FastAPI Swagger OpenAPI Documentation at /docs

Figure 11 showcases the backend OpenAPI / Swagger documentation at http://localhost:3000/docs, documenting all REST endpoints, request schemas, JWT bearer authorization headers, and multipart file upload models.

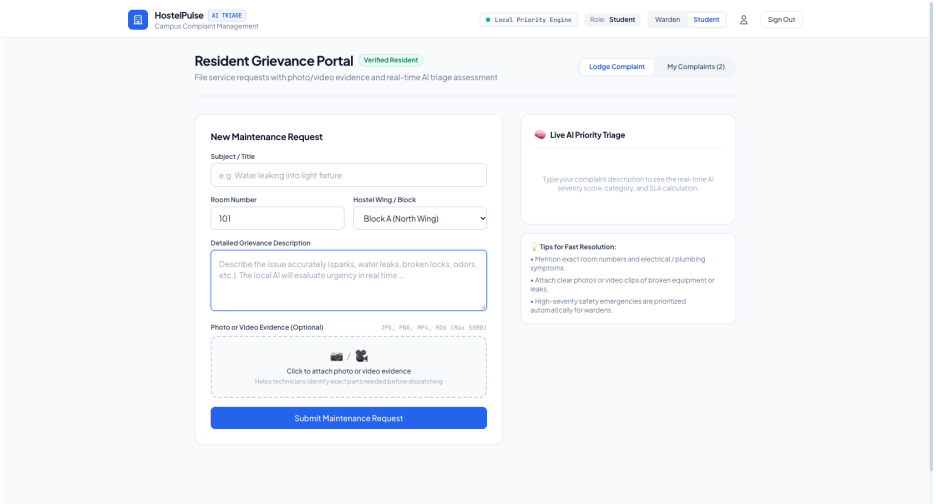


Figure 12: Student Grievance Submission with Photographic and Video Evidence Upload

Figure 12 illustrates the multi-media upload interface. Students can upload photos and video evidence to eliminate contractor ambiguity and furnish wardens with tangible visual proof.

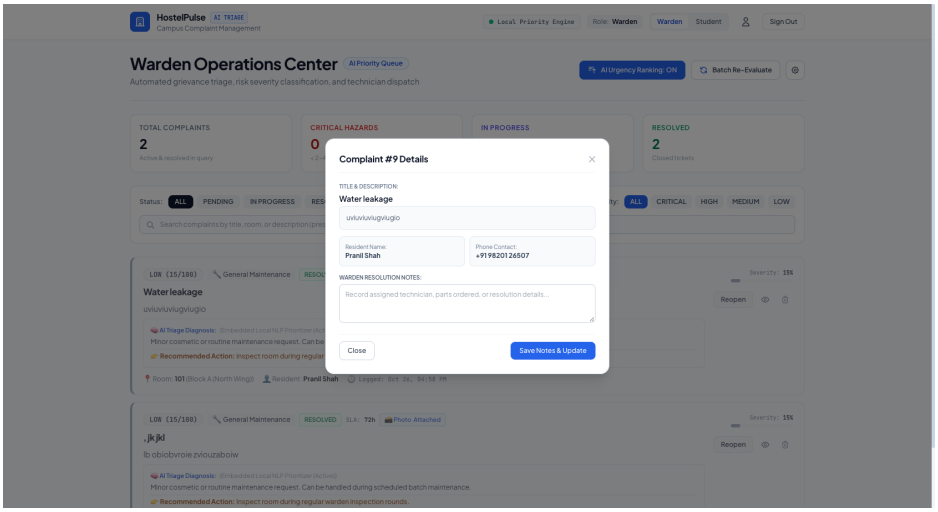


Figure 13: Warden Multi-Media Evidence Inspection Modal with Full-Resolution Image and Video Playback

Figure 13 demonstrates the warden evidence telemetry modal. Wardens can stream video recordings and inspect high-resolution photographs with native browser controls before dispatching maintenance staff.

8.2 Operational Usage Instructions

The following operational workflows guide students and hostel wardens in interacting with the system:

Student Operational Workflow:

- **1. Authentication:** Navigate to http://localhost:5173 and log in with student credentials, or click 'Student Demo (Pranil Shah)' for 1-click access.
- **2. Navigate to Lodge Complaint:** Click 'Lodge Grievance' in the navigation bar to open the reporting portal.
- **3. Formulate Description:** Type a description of the issue. Observe the 'Live AI Triage Preview' card dynamically update the predicted category and priority score.
- **4. Attach Proof:** Click 'Attach Media Evidence' to upload a photograph (.png, .jpg) or video recording (.mp4).
- **5. Submit & Track:** Click 'Submit Grievance'. Navigate to 'My Complaints' to inspect real-time progress on the timeline tracker.

Warden Operational Workflow:

- **1. Access Dashboard:** Sign in with warden credentials, or click 'Chief Warden' on the login screen.
- **2. Triage Queue:** Inspect the complaint queue. Notice that critical emergencies (Score >= 80) are pinned at the top.
- **3. Inspect Evidence:** Click on any complaint card to view the student's attached photo or stream video evidence.
- **4. Update Lifecycle:** Transition status from 'Pending' to 'In Progress' when technicians are dispatched, and to 'Resolved' upon repair.
- **5. Configure AI Engine:** Click 'AI Settings' in the header to switch between local Ollama models or inspect heuristic rules.

9. Conclusion

The Intelligent Hostel Complaint Management System successfully demonstrates how modern Database Management System (DBMS) principles can be synergized with on-premise Large Language Models to solve critical campus infrastructure challenges. By replacing paper registers with a normalized, indexed relational database, the system eliminates lost complaints, enforces referential integrity, and provides sub-50ms query performance.

The integration of local LLM priority triage fundamentally shifts hostel administration from arbitrary First-Come First-Served processing to risk-driven triage. Life-safety hazards such as electrical sparking and severe flooding are escalated immediately to wardens with computed priority scores (0-100), while non-critical repairs are queued systematically. Furthermore, by hosting quantized models locally via Ollama with zero-cloud API calls, the system guarantees 100% student data privacy and eliminates recurring commercial software licensing costs.

The multi-media evidence pipeline provides irrefutable ground truth, eliminating contractor disputes and reducing technician turnaround time. The clean, professional white aesthetic ensures an intuitive user experience for both students and hostel staff.

Future Enhancements include: (1) An automated WhatsApp bot dispatching notifications to maintenance technicians; (2) A technician mobile application with QR-code room scanning to verify on-site arrival; (3) Predictive maintenance analytics forecasting plumbing and electrical fixture failures based on historical seasonal trends.

10. References

• **[1] FastAPI Framework Documentation:** Tiangolo, S. (2024). FastAPI: High-performance modern Python web framework. Available at: https://fastapi.tiangolo.com/

• **[2] SQLite 3 Database Engine:** Hipp, D. R. (2024). SQLite: An In-Process Relational Database Engine. Available at: https://www.sqlite.org/docs.html

• **[3] Ollama Local LLM Runner:** Ollama Team (2024). Get up and running with large language models locally. Available at: https://ollama.com/

• **[4] Database System Concepts (7th Ed.):** Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts. McGraw-Hill Education, New York.

• **[5] React 18 Library Documentation:** Meta Open Source (2024). React: A JavaScript library for building user interfaces. Available at: https://react.dev/

• **[6] Vite Next Generation Frontend Tooling:** You, E. (2024). Vite: Next Generation Frontend Tooling. Available at: https://vitejs.dev/

• **[7] Tailwind CSS Documentation:** Wathan, A. (2024). Tailwind CSS: A utility-first CSS framework. Available at: https://tailwindcss.com/

• **[8] Python Cryptographic Libraries (Passlib & PyJWT):** Curchod, E. et al. (2024). Password hashing and JSON Web Tokens in Python. Available at: https://passlib.readthedocs.io/